

1 Изучение предметной области и анализ требований к LMS-платформам

- **1.1 Анализ предметной области: структура и участники учебного процесса в высшем учебном заведении**
- 1.2 Обзор и сравнительный анализ существующих систем управления обучением (LMS)
- 1.3 Анализ лучших практик организации интерфейсов и пользовательского опыта в образовательных сервисах
- 1.4 Разработка функциональных требований к клиентской части платформы
- 1.5 Разработка нефункциональных требований (*производительность, масштабируемость, безопасность, интернационализация, доступность*)
- **1.6 Проектирование и описание пользовательских сценариев**

1.1 Анализ предметной области: структура и участники учебного процесса в высшем учебном заведении

1.1.1 Терминологический глоссарий предметной области

В рамках автоматизируемой образовательной среды ключевые сущности учебного процесса формализуются следующим образом.

Пользователь (User) — субъект системы, обладающий уникальными учётными данными для входа и назначенной ролью, которая определяет набор доступных интерфейсов и операций. Система поддерживает три основные роли: Администратор, Преподаватель и Студент.

Роль (Role) — именованный набор разрешений, закреплённый за пользователем. На уровне клиентского приложения роль управляет видимостью компонентов навигации, доступностью маршрутов и составом отображаемых элементов управления. Проверка роли осуществляется на клиенте на основе декодирования JWT-токена, полученного при аутентификации.

Электронный учебный курс (Course) — структурированная единица образовательного контента, выступающая в роли контейнера для учебных материалов, метаданных и пользователей. На уровне клиентского приложения курс представляется в виде иерархического дерева тем и элементов с возможностью перехода между разделами без перезагрузки страницы.

Элемент курса (Course Element) — минимальная неделимая единица учебного контента. Сущность полиморфна: на клиенте реализуются типы «видеоматериал», «PDF-документ», «презентация» и «тест», каждый из которых требует собственного компонента отображения.

Банк вопросов (Question Bank) — изолированное логическое хранилище тестовых и контрольных заданий, категоризованных по темам, уровням сложности и типам ответов. Используется преподавателем для статической или динамической сборки тестов. Архитектурная изолированность банка позволяет переиспользовать вопросы в нескольких тестах без дублирования.

Тест (Assessment) — контрольное мероприятие, собранное из вопросов банка. Тест обладает набором параметров прохождения: ограничение по времени, срок сдачи, проходной балл. На клиенте тест порождает сессию прохождения.

Сессия тестирования (Assessment Session) — временное состояние, возникающее в момент начала прохождения теста студентом. Сессия хранит: идентификатор

студента, снимок вопросов, текущие ответы, оставшееся время и статус завершения.

Оценка (Grade) — количественный результат прохождения теста или выполнения задания. Для тестов с выбором ответа оценка вычисляется автоматически сервером; для заданий со свободным ответом требует ручной проверки преподавателем.

Балльно-рейтинговая система (БРС) — накопительная модель оценки успеваемости, агрегирующая оценки за все элементы курса в итоговый рейтинг студента. Требует от клиентского интерфейса динамических аналитических дашбордов и интерактивных ведомостей для преподавателей.

Академическая группа (Academic Group) — совокупность пользователей с ролью «Студент», объединённых общим годом поступления и направлением подготовки. Закрепление группы за курсом определяет, для каких студентов курс будет отображаться в их личных кабинетах.

1.1.2 Организационно-структурная модель и её отображение в SPA-архитектуре

Организационная структура университета имеет строго иерархический характер. С точки зрения информационного моделирования вложенность сущностей вуза описывается следующей цепочкой:

Университет → Институт / Факультет → Кафедра → Направление
подготовки → Академическая группа → Студент

Данная иерархия напрямую проецируется на архитектуру Single Page Application, определяя структуру клиентского роутинга и интерфейсные паттерны.

Динамический клиентский роутинг. Иерархия вуза задаёт вложенную структуру динамических маршрутов в менеджере маршрутов. Каждый уровень иерархии соответствует сегменту URL с параметром-идентификатором, что позволяет компонентам страниц извлекать нужный идентификатор из параметров маршрута и выполнять соответствующий API-запрос без передачи данных через глобальное хранилище.

Древовидные компоненты интерфейса. Для визуализации иерархии в административной панели применяются интерактивные древовидные компоненты. Они позволяют администратору наглядно воспринимать связи между кафедрами, направлениями и группами, а также выполнять операции добавления, перемещения или удаления узлов в рамках единого экрана.

Каскадный выбор. При создании учётных записей или закреплении преподавателей за структурными подразделениями используются каскадные выпадающие списки.

Клиентское приложение динамически фильтрует список дочерних элементов при выборе родительского: выбор института ограничивает список кафедр, выбор кафедры — список направлений. Это снижает вероятность ошибок ввода и уменьшает число запросов к серверу.

Изолированные порталы по ролям. Каждая роль пользователя обслуживается собственным набором защищённых маршрутов. Попытка перехода к маршруту другой роли вызывает автоматическое перенаправление. Такая изоляция делает порталы независимыми модулями, развиваемыми параллельно.

1.1.3 Жизненный цикл и состояния ключевых сущностей

Учебный процесс представляет собой совокупность динамических процессов. Для обеспечения корректного поведения интерфейса состояния ключевых сущностей описываются в виде конечных автоматов: текущее состояние управляет доступностью элементов управления и логикой отображения данных.

Состояния курса:

- **Черновик (Draft)** — начальное состояние при создании. Курс доступен для редактирования преподавателю, но полностью скрыт от студентов.
- **Активный (Active)** — курс опубликован. Отображается на дашборде студентов прикрепленных групп, открывается доступ к материалам и тестам.
- **Архивный (Archived)** — курс завершён. Для студентов переходит в режим «только чтение», убирается с главного дашборда и перемещается в раздел истории.

Состояния задания (практической работы):

- **Не начато (Not Started)** — задание доступно, кнопка отправки активна.
- **В процессе (In Progress)** — студент сохранил черновик ответа.
- **На проверке (Pending Review)** — решение отправлено. У студента блокируется повторная отправка; у преподавателя запись подсвечивается как требующая проверки.
- **Проверено (Graded)** — преподаватель выставил оценку и оставил комментарий. Редактирование заблокировано для обеих сторон.

Состояния тестовой сессии:

- **Доступен (Available)** — тест опубликован, кнопка запуска активна.
- **Выполняется (In Session)** — студент запустил тест. Активирован таймер обратного отсчета;
- **Завершён (Submitted)** — время вышло или студент нажал «Завершить». Данные отправлены на сервер, запускается автоматическая проверка.
- **Оценен (Graded)** — результаты доступны студенту.

1.2 Обзор и сравнительный анализ существующих систем управления обучением (LMS)

Для обоснованного выбора архитектурных решений и интерфейсных паттернов необходимо провести анализ существующих систем управления обучением. Среди платформ, широко применяемых в академической среде, наибольший интерес представляют четыре системы: Moodle, Canvas LMS и Stepik. Они охватывают спектр от классических монолитных решений с открытым исходным кодом до современных высоконагруженных облачных SaaS-платформ.

Moodle

Moodle (Modular Object-Oriented Dynamic Learning Environment) — наиболее распространённая платформа в российских вузах. Это монолитная система с открытым исходным кодом, написанная на PHP. Главное достоинство Moodle — развитая экосистема плагинов и глубокая проработка академических сценариев: ведение журналов, учёт групп и иерархических структур вуза, интеграция с деканатом.

Однако с точки зрения фронтенд-архитектуры Moodle имеет существенные недостатки. Система использует классический серверный рендеринг на основе шаблонизатора Mustache и устаревших JS-библиотек (jQuery, RequireJS). Это приводит к полной перезагрузке страницы при каждом переходе, медленному отклику интерфейса и перегруженному DOM-дереву. Реализация плавных анимаций и интерактивного тестирования в реальном времени в рамках такой архитектуры сопряжена со значительными трудозатратами.

Ценный опыт: детальная ролевая модель и полнофункциональная поддержка иерархических структур вуза (факультет → кафедра → группа → студент).

Canvas LMS

Canvas LMS от компании Instructure построен на базе Ruby on Rails и использует React в качестве фронтенд-стека. Платформа предлагает более отзывчивый и визуально чистый интерфейс по сравнению с Moodle: часть экранов реализована как React SPA внутри монолитного приложения.

Тем не менее клиентская часть Canvas глубоко интегрирована в серверный монолит, что усложняет кастомизацию. Самостоятельное развёртывание Canvas LMS представляет собой сложную инженерную задачу из-за жёсткой привязки к облачной инфраструктуре AWS. Платформа ориентирована на западный SaaS-рынок и слабо поддерживает особенности российской академической среды.

Ценный опыт: применение компонентного подхода на базе React для отдельных интерактивных экранов; концепция «модули → задания → баллы», аналогичная требуемой БРС.

Stepik

Stepik — современная российская облачная платформа, ориентированная на интерактивные курсы и автоматическую проверку задач. Клиентская часть реализована как высокопроизводительное SPA на базе React. Stepik демонстрирует эталонный пользовательский опыт: мгновенные переходы между страницами, плавно работающий таймер тестов, удобный конструктор шагов курса.

Принципиальное ограничение — исключительно SaaS-модель без возможности развёртывания на серверах университета, что исключает интеграцию с внутренними базами данных вуза и соответствие требованиям законодательства о персональных данных. Кроме того, в Stepik отсутствуют инструменты администрирования сложной вузовской структуры (кафедры, направления, закреплённые академические группы).

Ценный опыт: SPA-архитектура клиента с мгновенной навигацией; продуманный дизайн интерфейса прохождения теста с картой вопросов и таймером.

Сравнительная таблица

Таблица 1.1 — Сравнительный анализ архитектуры и возможностей существующих LMS

Критерий сравнения	Moodle	Canvas LMS	Stepik
Архитектура клиента	SSR (PHP + Mustache)	Гибридная (Rails + React)	SPA (React)

Скорость навигации	Низкая (полная перезагрузка)	Средняя	Высокая (без перезагрузки)
Адаптивность (mobile)	Удовлетворительная	Хорошая	Отличная
Возможность on-premise	Полная	Затруднённая	Отсутствует
Поддержка иерархии вуза	Полная	Частичная	Ограниченная
Ролевая модель	Развитая	Средняя	Упрощённая
Тип лицензии	Open Source (GPL)	Open Source / SaaS	SaaS (закрытый)
Автосохранение сессии теста	Частичное	Есть	Есть

Выводы и заимствованные практики

Проведенный анализ позволяет выделить четыре направления лучших практик, которые служат ориентирами при проектировании разрабатываемой платформы.

1. SPA-архитектура клиента (опыт Stepik). Все сценарии взаимодействия, требующие высокой отзывчивости — прохождение теста, редактирование содержания курса, навигация по структуре — реализуются на клиенте без полной перезагрузки страницы. Vue Router обеспечивает мгновенные переходы между экранами внутри одного SPA.

2. Глубокая ролевая модель и поддержка иерархии вуза (опыт Moodle). Система учитывает реальную структуру российского вуза: университет → институт → кафедра → направление → группа → студент. Закрепление пользователей за структурными подразделениями определяет область видимости контента в интерфейсе каждой роли.

3. Изолированные компоненты для типов контента (опыт Canvas). Каждый тип элемента курса (видео, PDF, тест, задание) реализуется как самостоятельный полиморфный компонент с собственной логикой отображения, не зависящей от соседних типов.

4. Надёжная сессия тестирования с автосохранением (опыт Stepik).

Промежуточные ответы студента автоматически сохраняются на сервер по мере

заполнения. При случайном закрытии вкладки или сетевом сбое сессия восстанавливается с последнего сохранённого состояния.

При этом цель данной работы не состоит в создании прямой альтернативы перечисленным коммерческим и академическим решениям. Основная научно-практическая задача заключается в прохождении полного жизненного цикла разработки сложной образовательной веб-платформы с применением современного технологического стека и методологий.

1.4 Разработка функциональных требований к клиентской части платформы

Функциональные требования определяют то, какие возможности должна предоставлять система пользователям. В ходе анализа были формализованы и систематизированы функциональные требования для трех ключевых ролей пользователей: Администратора, Преподавателя и Студента.

Ниже приведено описание функциональных требований к платформе к интерфейсу каждого участника образовательного процесса.

Общие функциональные требования

Перед началом работы с платформой клиентская часть должна обеспечивать прохождение сквозных процедур, обязательных для всех категорий пользователей:

- **Идентификация и аутентификация:** каждый пользователь системы должен проходить процедуру авторизации с помощью пары уникального логина и пароля.
- **Разграничение прав доступа:** после успешного входа система определяет назначенную роль пользователя и на клиентском уровне динамически формирует доступный набор интерфейсов, меню навигации и защищенных маршрутов.
- **Сброс и восстановление пароля:** клиентская часть должна содержать сценарий самостоятельного восстановления пароля пользователем через указание электронной почты, связанной с учетной записью, и последующее подтверждение изменений.

Функциональные требования к интерфейсу Администратора

Интерфейс Администратора ориентирован на техническое обеспечение работоспособности системы, управление иерархической структурой вуза и ведение базы данных пользователей:

- **Управление учетными записями пользователей:** Администратор должен иметь возможность создавать новые учетные записи (как вручную для единичных пользователей, так и пакетно посредством импорта файлов в формате Excel), редактировать существующие профили (изменять электронную почту, переназначать группы) и удалять неактуальные аккаунты.
- **Управление иерархической структурой университета:** интерфейс должен предоставлять инструменты для визуального редактирования древовидной структуры направлений подготовки, институтов/факультетов, кафедр и академических групп.
- **Управление правами доступа преподавателей:** Администратор закрепляет за конкретным преподавателем список доступных ему учебных направлений, институтов или групп, определяя его зону ответственности в системе.

Функциональные требования к интерфейсу Преподавателя

Интерфейс Преподавателя предназначен для создания учебного контента, контроля успеваемости студентов и управления учебными курсами:

- **Управление курсами:** возможность создания новых курсов с указанием названия, описания и закрепленных академических групп студентов. Преподаватель может переводить курсы в архивный статус и добавлять других преподавателей в качестве соавторов.
- **Конструирование содержания:** интерфейс позволяет выстраивать структуру курса и наполнять ее учебными материалами разных форматов (текст, презентации, PDF-документы, видеофайлы).
- **Управление банком вопросов:** создание и ведение централизованной базы вопросов различных типов (тестовые с одиночным или множественным выбором, контрольные со свободным кратким или развернутым ответом). Вопросам могут назначаться параметры сложности.
- **Создание тестов и контрольных заданий:** сборка тестов из готовых банков вопросов, установка проходного балла, ограничения по времени на прохождение и жестких сроков сдачи.
- **Мониторинг успеваемости студентов:** Преподавателю предоставляется детальная статистика по курсу в виде интерактивных графиков и таблиц, отражающих прогресс прохождения тем, результаты тестирования и общую

успеваемость групп. Должен быть реализован экспорт этих отчетов в файлы формата Excel.

- **Проверка работ:** интерфейс должен содержать удобный инструмент для ручной проверки развернутых ответов студентов на контрольные задания с возможностью выставления оценки и отправки текстового комментария.

Функциональные требования к интерфейсу Студента

Интерфейс Студента представляет собой адаптивное рабочее пространство, спроектированное для максимально комфортного и сфокусированного прохождения обучения:

- **Дашборд доступных курсов:** Студент должен видеть наглядный перечень всех учебных курсов, на которые он записан.
- **Взаимодействие с учебным материалом:** навигация по структуре курса, последовательное чтение лекций, просмотр прикрепленных видео, а также предпросмотр и скачивание методических материалов (PDF, презентаций).
- **Прохождение тестовых работ:** интерактивное выполнение тестов (с визуальным отображением оставшегося времени) и отправка решений практических или контрольных работ (включая загрузку файлов с выполненными заданиями).
- **Мониторинг успеваемости:** доступ к разделу с собственными оценками по всем элементам курса, обратной связью от преподавателей и общим индикатором прогресса прохождения учебной программы.

1.5 Разработка нефункциональных требований

Нефункциональные требования определяют качественные характеристики клиентской части платформы, накладывая ограничения на архитектурные решения, производительность, безопасность и пользовательский опыт. Специфика разработки клиентского SPA-приложения для академической среды обуславливает следующие нефункциональные требования:

Требования к интерфейсу пользователя

- **Интуитивность и доступность:** интерфейс должен быть интуитивно понятным для пользователей с разным уровнем компьютерной грамотности. Навигация должна строиться по принципу минимального количества кликов для достижения целевого действия.
- **Кроссплатформенная адаптивность:** клиентская часть должна корректно отображаться и сохранять полную функциональность на широком спектре устройств: персональные компьютеры, ноутбуки, планшеты.

- **Доступность:** обязательна поддержка стандартов доступности для людей с ограниченными возможностями здоровья (контрастные цветовые схемы, масштабирование текста без разрушения верстки, поддержка навигации с клавиатуры).

Производительность и оптимизация (Performance)

- **Время первичной загрузки:** размер исходного бандла приложения должен быть минимизирован за счет разделения кода и ленивой загрузки некритичных модулей. Время до первой отрисовки контента (FCP) при стандартном интернет-соединении должно составлять не более 1.5–2 секунд.
- **Отзывчивость интерфейса:** переходы между страницами внутри Single Page Application должны происходить плавно, без визуального мерцания и полной перезагрузки страниц. Для сглаживания времени ожидания ответов от сервера необходимо использовать анимированные скелетоны.
- **Управление состоянием:** хранение глобального состояния приложения должно исключать лишние повторные сетевые запросы при переключении между вкладками или возврате на предыдущие страницы.

Безопасность и разграничение доступа

- **Защита каналов связи:** весь обмен данными между клиентской и серверной частями платформы должен осуществляться по защищенному протоколу HTTPS с использованием современных алгоритмов шифрования.
- **Безопасное хранение сессий:** авторизационные токены (JWT) должны храниться в защищенных клиентских хранилищах, минимизирующих риски XSS-атак (рекомендуется использование HttpOnly Cookie или защищенного стейта приложения).
- **Строгая изоляция данных:** на клиентском уровне должна быть жестко заблокирована возможность просмотра данных, не принадлежащих текущему пользователю. Студент может видеть только свои оценки, а Преподаватель — только данные студентов, привязанных к его курсам. Изменение персональных данных пользователей и иерархии вуза должно быть полностью недоступно в интерфейсе Преподавателя и Студента.

Надежность и отказоустойчивость

- **Обработка сетевых ошибок:** в случае кратковременного обрыва связи или недоступности API бэкенда клиентская часть должна корректно обрабатывать эти ситуации (выводить понятные пользователю предупреждения с предложением

повторить попытку), предотвращая зависание интерфейса или потерю несанкционированно введенных данных (например, ответов на вопросы теста).

- **Автоматическое сохранение прогресса:** при прохождении студентом тестирования его промежуточные ответы должны автоматически кэшироваться или отправляться на сервер, чтобы в случае случайного закрытия вкладки или сбоя компьютера студент мог продолжить выполнение задания с того же места.

1.6 Проектирование и описание пользовательских сценариев

Для детальной спецификации функциональных требований и последующего проектирования интерфейсов необходимо формализовать основные сценарии взаимодействия пользователей с платформой. Каждый сценарий описывает последовательность действий актора и реакцию системы, определяя прецеденты использования клиентской части приложения.

1.6.1 Сценарии взаимодействия для роли «Администратор»

Сценарий 1.6.1.1: Ручное создание учетной записи пользователя

- **Предусловия:** Администратор авторизован в системе и находится в административной панели.
- **Основной сценарий (Шаги):**
 1. Администратор переходит на страницу «Управление пользователями».
 2. Нажимает кнопку «Создать пользователя».
 3. Заполняет интерактивную форму ввода данных (ФИО, уникальный логин, email, выбор роли: Администратор, Преподаватель, Студент).
 4. Нажимает кнопку «Сохранить».
- **Результат:** Данные отправляются на сервер, в базе данных создается запись, а клиентская часть отображает уведомление об успешном создании.

Сценарий 1.6.1.2: Импорт учетных записей из внешнего файла

- **Предусловия:** Администратор подготовил файл со списком пользователей в формате Excel.
- **Основной сценарий (Шаги):**
 1. Администратор переходит в раздел «Управление пользователями».
 2. Нажимает кнопку «Импортировать из файла».
 3. В диалоговом окне выбирает подготовленный файл и нажимает «Загрузить»..
 4. Администратор подтверждает действие нажатием кнопки «Импортировать».
- **Результат:** Пакет данных отправляется на сервер, в системе регистрируются новые пользователи, интерфейс выводит отчет об успешном импорте и количестве созданных записей.

Сценарий 1.6.1.3: Модификация организационной структуры вуза

- **Предусловия:** Администратор находится в разделе «Иерархия структуры».
- **Основной сценарий (Шаги):**
 1. Администратор просматривает древовидную структуру направлений подготовки, институтов и групп.
 2. Выбирает необходимый узел дерева (например, конкретный Институт) и нажимает кнопку «Добавить дочерний элемент» или «Редактировать».
 3. Вводит новое название (например, название создаваемой академической группы) и сохраняет изменения.
- **Результат:** Дерево структуры динамически перерисовывается на клиенте, отправляя обновленную конфигурацию иерархии на сервер.

Сценарий 1.6.1.4: Просмотр и редактирование профиля пользователя

- **Предусловия:** Пользователь зарегистрирован в системе.
- **Шаги:**
 1. Администратор переходит на страницу «Пользователи».
 2. Использует фильтры и строку поиска для нахождения нужного человека.
 3. Кликает на имя пользователя для открытия карточки профиля.
 4. Вносит необходимые изменения (корректирует email, переназначает учебную группу или направление).
 5. Нажимает кнопку «Сохранить изменения».
- **Результат:** Обновленная информация отправляется на сервер, локальный стек данных на клиенте обновляется, выводится уведомление об успешном сохранении.

1.6.2 Сценарии взаимодействия для роли «Преподаватель»

Сценарий 1.6.2.1: Создание учебного курса

- **Предусловия:** Преподаватель авторизован в системе.
- **Шаги:**
 1. На главной странице нажимает кнопку «Добавить курс».
 2. Вводит название и подробное описание курса.
 3. При необходимости указывает соавторов курса из числа других преподавателей.
 4. Нажимает кнопку «Сохранить изменения».
- **Результат:** Курс создается и становится доступным для редактирования и наполнения в личном кабинете преподавателя.

Сценарий 1.6.2.2: Структурирование содержания курса

- **Предусловия:** Учебный курс успешно создан.
- **Шаги:**
 1. Преподаватель переходит в раздел «Содержание курса».
 2. Нажимает кнопку добавления новой темы (например, «Раздел 1»).
 3. Внутри созданной темы нажимает «Добавить подтему».
 4. Нажимает кнопку «Сохранить изменения».
- **Результат:** На клиенте формируется иерархическая структура курса, готовая к наполнению материалами.

Сценарий 1.6.2.3: Добавление учебных материалов в курс

- **Предусловия:** Структура курса (разделы и подразделы) сформирована.
- **Шаги:**

1. Преподаватель выбирает нужную подтему в структуре курса.
 2. Нажимает кнопку «Добавить материал».
 3. Выбирает тип материала (видео, презентация, PDF-документ, текстовый файл) и загружает его в систему.
 4. Указывает название материала и краткое описание, после чего нажимает «Сохранить».
- **Результат:** Материал прикрепляется к подтеме курса и становится доступен для изучения студентам.

Сценарий 1.6.2.4: Добавление вопросов в банк вопросов

- **Предусловия:** Преподаватель находится в разделе «Банк вопросов».
- **Шаги:**
 1. Выбирает существующий банк из списка или создает новый нажатием на «Добавить банк вопросов».
 2. Внутри выбранного банка нажимает кнопку «Добавить вопрос».
 3. Определяет тип вопроса (тестовый с вариантами, контрольный с текстовым вводом).
 4. Заполняет текст вопроса и указывает сложность.
 5. Для тестового вопроса добавляет варианты ответов и отмечает правильные; для программного — указывает критерии оценки и тесты.
 6. Нажимает «Сохранить вопрос».
- **Результат:** Вопрос заносится в базу данных банка вопросов и готов к использованию в тестах.

Сценарий 1.6.2.5: Создание теста на основе банка вопросов

- **Предусловия:** Создана подтема курса, сформирован и наполнен банк вопросов.
- **Шаги:**
 1. Преподаватель переходит в нужную подтему курса и нажимает «Добавить тест».
 2. Вводит название теста, описание и параметры прохождения (время выполнения, дедлайны).
 3. Переходит во вкладку «Вопросы» и нажимает «Выбрать вопросы из банка».
 4. Фильтрует вопросы по типу и сложности, отмечает галочками нужные и импортирует их.
 5. Настраивает количество начисляемых баллов за каждый верный ответ.
 6. Нажимает «Сохранить тест».
- **Результат:** Тест публикуется в структуре курса и становится доступен для прохождения студентам.

Сценарий 1.6.2.6: Проверка контрольных заданий студентов

- **Предусловия:** Студенты выполнили контрольные работы, требующие ручной оценки.
- **Шаги:**
 1. Преподаватель переходит в раздел статистики и успеваемости курса.
 2. Выбирает необходимое задание из списка требующих проверки.
 3. Просматривает прикрепленные файлы решений студентов.
 4. Выставляет оценку в баллах (вручную) и при необходимости пишет комментарий с разбором ошибок.
 5. Нажимает «Сохранить оценки».
- **Результат:** Оценки фиксируются в системе, студент получает уведомление, а общая ведомость успеваемости автоматически пересчитывается.

Сценарий 1.6.2.7: Отслеживание успеваемости и экспорт отчетов

- **Предусловия:** Студенты начали прохождение курса.
- **Шаги:**
 1. Преподаватель переходит в раздел статистики курса.
 2. Выбирает академическую группу или конкретного студента для анализа.
 3. Изучает сводный прогресс прохождения тем и результаты сданных тестов.
 4. Нажимает кнопку «Экспорт».
 5. Выбирает желаемый формат файла (CSV или Excel) и скачивает его на устройство.
- **Результат:** На устройство скачивается файл с актуальными данными об успеваемости студентов.

Сценарий 1.6.2.8: Назначение учебных групп на курс

- **Предусловия:** Курс создан, Администратор назначил преподавателя на соответствующие группы.
- **Шаги:**
 1. Преподаватель заходит в настройки нужного курса.
 2. Переходит во вкладку «Доступ групп».
 3. Из предложенного перечня доступных ему групп отмечает галочками те, для которых открывается данный курс.
 4. Нажимает кнопку «Сохранить изменения».
- **Результат:** Курс появляется в личных кабинетах всех студентов выбранных академических групп.

1.6.3 Сценарии взаимодействия для роли «Студент»

Сценарий 1.6.3.1: Прохождение разделов учебного курса

- **Предусловия:** Студент авторизован на платформе и записан на курс.
- **Шаги:**
 1. Студент открывает список своих курсов и переходит на нужный курс.
 2. Система выводит структуру доступных материалов.
 3. Студент выбирает лекцию или практический материал для изучения.
 4. В зависимости от типа контента: изучает текст на странице, просматривает видеоматериал или скачивает прикрепленный PDF-документ (презентацию).
 5. Использует систему навигации для быстрого перехода к следующей теме курса.
- **Результат:** В системе фиксируется факт изучения темы, прогресс прохождения курса студентом увеличивается.

Сценарий 1.6.3.2: Прохождение тестовой работы

- **Предусловия:** Студенту доступен опубликованный в курсе тест.
- **Шаги:**
 1. Студент открывает список материалов курса и выбирает активный тест.
 2. Ознакомляется с параметрами (количество попыток, время) и нажимает кнопку начала тестирования.
 3. Последовательно отвечает на вопросы, выбирая варианты ответов или вписывая краткие ответы с клавиатуры.
 4. Контролирует оставшееся время по таймеру на экране.
 5. Нажимает кнопку «Завершить тест».
- **Результат:** Система фиксирует ответы, отправляет их на автоматическую проверку и (в зависимости от настроек теста) сразу выводит набранные баллы и детальный отчет по ошибкам.